Supervised Deep Learning Under Label Noise: Reproduction of LabelWave and an Entropy-Aware Loss Extension

M. Muneeb Lone (23i-2623) Kabeer Ali (23i-2625) Ismail Kashif (23L-2549)
Department of Data Science, FAST-NUCES, Islamabad, Pakistan

Abstract. The challenge associated with label noise lies in that it can often remain unnoticed until it damages a model. Since conventional cross-entropy training does not distinguish between a proper label and a noisy one, the latter is memorized along with the former by the network. The paper provides two contributions related to the problem discussed above.

As part of our first contribution, we replicate LabelWave [1], an ICLR 2024 technique, which measures how often a model changes the predicted class of individual training samples between consecutive epochs and stops learning when such instability increases sharply. For evaluation purposes, we use CIFAR-10 corrupted by 40% symmetric noise; training runs on 50 CPU epochs. Our reproduction reaches a peak test set accuracy of 83.42%, which actually exceeds the paper's reported 78.4% after 200 GPU epochs. The reasons behind this outcome are described below.

Secondly, we propose and implement an entropy-aware extension called Entropy-Aware Confidence-Penalized Loss (EA-CPL). In order to apply it, the amount of label smoothing for each sample is made adaptive based on the prediction entropy of the corresponding softmax output. High-confidence predictions are penalized heavily to dampen the loss function gradient caused by possibly erroneous labels. The proposed EA-CPL is benchmarked against a cross-entropy baseline trained on the same noisy dataset using the same network architecture, optimizers and number of epochs on a Tesla T4 GPU. EA-CPL scores 79.00% on the test set compared to 79.94% of the baseline, thus, achieving a relative improvement of -0.94%. The explanation of this result is provided next.

1. Introduction

Every deep learning pipeline implicitly assumes that the labels used in training are correct for the majority of examples. Unfortunately, this is not always the case. Massive datasets are labeled by crowdsourcing workers who sometimes hurry to complete the task, disagree with each other, or even get confused by genuinely ambiguous samples. Even carefully crafted datasets from academia are far from perfect. For example, Northcutt et al. [5] showed that pervasive labeling errors exist in popular benchmark test sets, large enough to affect the ranking of competing approaches. It is hard to overestimate the severity of this problem since it means that a method that outperforms its competitors is just better at dealing with their mistakes during annotation rather than being more effective per se.

What is worse, the label errors are far from being uniformly distributed among all examples. Human annotators make mistakes more often when they have to deal with complex or ambiguous inputs. As a result, the label noise in a realistic training set is not uniform; rather, it accumulates in places where the model is less sure about classification. This poses two challenges simultaneously: the samples that are hard to classify correctly are precisely those that have a higher chance of having noisy labels. We inject artificial symmetric noise into the training set in our experiment to compare our results with the literature; yet, it is crucial to realize that real-life noise is much more structured.

The fundamental problem here is that standard training provides the model with no means to withstand bad labels even though it can detect them. Zhang et al. [4] demonstrated formally that modern deep networks have more than enough capacity to perfectly memorize completely random labels if trained for long enough. The network does not question whether a label is reasonable. It just tries to reduce the loss, and standard cross-entropy gives it every gradient update it needs to become increasingly confident in whatever label it receives, whether that label is correct or not. The problem is not that networks are too simple to recognize noise. It is that nothing in the ordinary training objective asks them to try.

Several lines of work have developed to address this at different points in the training pipeline. Some approaches modify the loss function directly, designing objectives that are mathematically more resistant to incorrect targets [8]. Others identify and remove suspicious training

examples before they can influence gradient updates, relying on the observation that clean examples tend to have lower loss in the early stages of training [6]. Curriculum-based approaches control the order in which training examples are presented, ensuring the model first learns clean, easy patterns before encountering harder and potentially corrupted ones [7]. More recently, researchers have explored interventions that operate at the level of the training loop itself, detecting the onset of memorization and stopping before it causes significant damage, without requiring access to any clean validation data [1–3].

The paper we reproduce, LabelWave [1], belongs to this third category of work. The key message in this case can be succinctly stated as follows. As the network gets increasingly involved in memorizing noisy labels, the predictions it makes per each training example become unpredictable. The model switches the class it predicts for individual samples between epochs as a result of conflicting changes in the direction of shifting parameters caused by corrupt labels in different batches. The degree of instability in the predictions is quantified using a single scalar variable – the label wave count, defined as the number of examples changing classes between consecutive epochs. A sharp increase in this count serves as a clear sign of memorization beginning, indicating that training needs to be stopped. The proposed approach is easy to implement as no additional components or modification of existing components are necessary; in particular, there is no need for a clean validation set, an auxiliary classifier, or a custom loss.

There are some limitations, however, associated with the early stopping procedure, which is at the core of the proposed method. Stopping the training process impacts all types of learning equally, including the type of learning driven by accurate labels. Thus, early stopping might impair the model's ability to extract discriminative features, which only emerge as a result of prolonged training in the presence of clean labels. To overcome this problem, we designed EA-CPL. The Entropy-Aware Confidence-Penalized Loss (EA-CPL) is our attempt to answer that question. The idea is that by scaling label smoothing per-sample based on prediction entropy, we can reduce the gradient contribution of overconfident, potentially noisy predictions without needing to identify or discard specific examples.

The rest of this report is organized as follows. Section 2 reviews the relevant prior work and describes how LabelWave and related methods fit into the broader landscape of robust learning under label noise. Section 3 describes the design and motivation of EA-CPL in detail, including the mathematical formulation and the architectural choices. Section 4 describes the experimental setup for both the reproduction and the extension. Section 5 reports the full results with tables and figures. Section 6 analyzes what the results mean, why EA-CPL did not outperform the baseline, and what the negative result implies about entropy-based noise proxies. Section 7 concludes and outlines directions for future work.

2. Background and Related Work

2.1 The Memorization Problem in Deep Networks

The foundation for most work in this area is the finding by Zhang et al. [4] that deep neural networks will converge even when trained on completely random labels, given a sufficient number of training epochs. This is not a flaw in any particular implementation. It is a direct consequence of the overparameterization that makes deep networks so effective in the first place. Standard cross-entropy loss treats every training label as ground truth. It provides a gradient signal for every example in every batch, and the network has both the capacity and the incentive to fit all of them, regardless of whether they reflect any meaningful structure in the input data.

The consequence in a real noisy-label setting is that the model will eventually memorize both the clean signal and the noise. Early in training, the model focuses on the dominant, statistically consistent patterns in the data, which mostly come from correctly labeled examples. This is the early learning phase where most robust methods try to extract useful information before the memorization phase takes over. In the memorization phase, the model starts fitting the inconsistent, noisy labels. Training loss continues to fall, but test accuracy plateaus or degrades because the model is now encoding noise rather than generalizable features. Northcutt et al. [5] made this concern concrete by showing that label errors in well-known benchmarks are substantial enough to affect evaluation, meaning the problem affects not just model training but how we measure progress in the field.

2.2 LabelWave: Stopping Without Validation Data

The paper we reproduce, by Yuan, Feng, and Liu [1], introduces a method called LabelWave that uses prediction instability as an early stopping criterion. The core observation is that during the early learning phase, the model's predictions on individual training samples stabilize relatively quickly as the network converges on the dominant clean patterns. During the memorization phase, predictions become unstable again because the noisy labels are inconsistent and pull the model's parameters in different directions on different training steps.

The label wave count $\mathcal{W}_t$ at epoch t is defined as:

$$\mathcal{W}_t = \sum_{i=1}^{N} \mathbf{1}\left[\hat{y}_i^{(t)} \neq \hat{y}_i^{(t-1)}\right] \tag{1}$$

where $\hat{y}_i^{(t)}$ is the predicted class for training sample i at epoch t and N is the total number of training samples. The count starts at zero because there is no previous epoch to compare against on the first pass. As training progresses, this count grows during the learning phase and then spikes sharply when memorization begins. Training is halted when the count crosses a threshold, which the authors set adaptively using the mean and standard deviation of the wave count over the first few epochs.

The authors validate LabelWave on CIFAR-10 and CIFAR-100 under symmetric and asymmetric noise at rates ranging from 20% to 60%. Across all settings, LabelWave consistently outperforms both validation-based early stopping and several other no-validation baselines. The method's practical advantage is significant: it requires no extra data, no additional model, and no hyperparameter specific to the noise level or the dataset. In our implementation, we track the label wave count by storing a dictionary that maps each sample's unique identifier to its predicted class from the most recent epoch. After each epoch, we compare current predictions against stored ones and increment the count for any mismatch. The memory overhead is one integer per training sample, and the computational overhead is negligible.

2.3 Robust Classification via Regression (ICLR 2024)

In this paper, Cheng et al. [2] consider a fundamentally different way to address the problem of label noise. Instead of changing the stopping criterion of the training process, the authors reformulate the classification objective as a regression problem, assuming a Gaussian distribution for the label noise and using it to simultaneously re-weight losses and correct labels. No sample selection procedure is required since the proposed approach works within a unified framework. In terms of performance, their experiments show strong empirical results on existing benchmarks with noisy labels. The major downside from the theoretical perspective concerns the Gaussian distribution assumption for noise: the idea does not apply if there is any semantic structure to the errors made or if they are localized in certain areas of the input space.

2.4 Conditional Distributionally Robust Optimization (NeurIPS 2024)

The paper by Liu et al. [3] describes a framework built on the basis of distributionally robust optimization. Instead of optimizing the model based on the expected loss from the potentially noisy label distribution, the authors propose to minimize the worst-case expectation of the loss, computed with respect to an ensemble of other possible distributions. These alternative distributions are obtained through likelihood ratio tests; furthermore, the resulting robust risk function has a closed-form expression and thus, theoretically, can be optimized effectively. The framework yields optimal theoretical guarantees on the worst-case risk and state-of-the-art performance when trained under extremely noisy labels. However, the main practical drawback concerns increased computational complexity compared to conventional training methods, making the approach impractical in cases like ours.

2.5 Symmetric Cross Entropy

This paper by Wang et al. [8] provides an important insight that contributed to our EA-CPL design. Traditional cross-entropy loss function is not symmetric since it only measures the Kullback-Leibler divergence of the model distribution from the one-hot target distribution but not vice versa. Thus, it is prone to overfit to difficult examples, which cause the model to generate extremely confident distributions. To address this issue, Wang et al. propose to add a penalty in the form of reversed KL divergence, leading to a symmetric cross-entropy loss that prevents overfitting without losing information from clean samples. Inspired by this finding, we extended it by designing adaptive smoothing with the current prediction entropy as scaling.

2.6 Sample Selection and Co-training Methods

Han et al. [6] introduced Co-teaching, a method that trains two independent networks simultaneously. At each step, each network selects the small-loss samples from the current mini-batch, assuming these are the clean examples, and passes them to the other network for gradient computation. The two-network setup prevents confirmation bias, where a single model reinforces its own errors. The small-loss heuristic exploits the empirical finding that clean examples tend to have lower training loss during the early learning phase, before the model has had time to memorize noisy labels.

Jiang et al. [7] take a curriculum approach. A mentor network, pre-trained on a small clean dataset, assigns weights to training examples before they are passed to the student network for learning. Examples that the mentor considers more likely to be clean receive higher weights. This curriculum guides the student away from corrupted labels, particularly early in training when the student is most susceptible to memorization.

Song et al. [9] provide a comprehensive and well-organized survey of all major approaches to learning with noisy labels in deep networks. Their taxonomy traces the evolution from early work on loss correction using estimated noise transition matrices, through sample selection and curriculum methods, to the more recent approaches based on semi-supervised learning, meta-learning, and distributionally robust optimization. It is a useful reference for understanding where any new method fits within the broader landscape.

3. Proposed Model: EA-CPL

3.1 Motivation and Core Idea

LabelWave handles label noise by watching the model and stopping it at the right time. Our proposed extension approaches the problem from the opposite direction. Instead of stopping early, we try to change what the model learns from throughout the entire training run. The goal is to design a loss function that naturally resists memorizing corrupted labels without requiring any explicit identification or filtering of noisy examples. If this works, then even without an early stopping criterion, the model should be less susceptible to the memorization problem that LabelWave is designed to detect.

The central observation behind EA-CPL is that standard cross-entropy encourages a form of confirmation bias. The model is driven to minimize its loss by becoming increasingly confident in the labels it sees. On a clean dataset, this is exactly what you want. But on a noisy one, it means the model will push toward extreme confidence on corrupted labels with the same intensity it would apply to correct ones. There is nothing in the loss that distinguishes the two cases. Once the model has updated its parameters enough times in the direction of a wrong label, it becomes very hard to correct. The gradient from that example keeps reinforcing the mistake.

The intuition behind EA-CPL is to introduce a per-sample penalty on overconfidence. When the model is highly certain about a prediction, we apply stronger label smoothing to that example, which softens the target distribution and reduces the gradient magnitude. When the model is uncertain, we leave the smoothing weaker and let the learning signal come through more fully. The reasoning is that high-confidence predictions on noisy examples are the primary driver of memorization. If we can suppress those specific gradient contributions without touching predictions where the model is appropriately uncertain, we might get a useful regularization effect.

We chose entropy as the confidence signal because it is a natural, well-defined measure of the spread of a probability distribution. It is easy to compute from the softmax output, it requires no extra forward passes, and it adds no parameters to the model. This makes EA-CPL a true drop-in replacement for standard cross-entropy, usable in any existing training pipeline without any architectural changes.

3.2 Entropy-Aware Confidence Estimation

For a training sample with output logits $\mathbf{z} \in \mathbb{R}^C$, we compute the softmax probability vector:

$$\mathbf{p} = \text{softmax}(\mathbf{z}), \quad p_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{2}$$

The Shannon entropy of this distribution is:

$$H(\mathbf{p}) = -\sum_{c=1}^{C} p_c \log(p_c + \varepsilon) \tag{3}$$

where C is the number of classes and $\varepsilon = 10^{-8}$ is a small constant to prevent numerical issues when p_c is very close to zero. We then normalize this entropy by the theoretical maximum, which is achieved when the distribution is perfectly uniform:

$$\kappa = 1 - \frac{H(\mathbf{p})}{\log C} \tag{4}$$

This gives a per-sample confidence score $\kappa \in [0, 1]$. When the model assigns equal probability to all C classes, $H(\mathbf{p}) = \log C$ and $\kappa = 0$. When the model is entirely concentrated on a single class, $H(\mathbf{p}) \approx 0$ and $\kappa \approx 1$. For intermediate cases, κ interpolates smoothly between these extremes. This normalization ensures that the confidence score is meaningful and comparable across different model states regardless of the number of classes.

3.3 Dynamic Per-Sample Label Smoothing

Given the confidence score κ_i for sample i, the per-sample smoothing factor is computed as:

$$\alpha_i = \text{clip}\big(\alpha_0 + \lambda \cdot \kappa_i,\ 0,\ 0.9\big) \tag{5}$$

where $\alpha_0 = 0.1$ is a base smoothing level applied universally to all samples, and $\lambda = 0.5$ is a penalty weight that controls how much the confidence score amplifies the smoothing. The clip operation keeps α_i in the range $[0, 0.9]$, ensuring the smoothed target never becomes completely flat (which would eliminate all class information) or fully one-hot (which would reduce EA-CPL to standard cross-entropy for low-confidence samples). In practice, with $\alpha_0 = 0.1$ and $\lambda = 0.5$, the smoothing factor ranges from 0.1 for a completely uncertain prediction to 0.6 for a maximally confident one.

The smoothed target distribution for sample i with given label y_i is:

$$\tilde{q}_{i,c} = \begin{cases} 1 - \alpha_i + \dfrac{\alpha_i}{C} & \text{if } c = y_i \\ \dfrac{\alpha_i}{C} & \text{if } c \neq y_i \end{cases} \tag{6}$$

This is standard label smoothing with a per-sample smoothing factor. When $\alpha_i = 0$, the target is one-hot. When $\alpha_i = 1$, the target is uniform across all classes. The EA-CPL loss is the average negative log-likelihood under these sample-specific soft targets:

$$\mathcal{L}_{\text{EA-CPL}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} \tilde{q}_{i,c} \log p_{i,c} \tag{7}$$

It is worth noting that the smoothed target $\tilde{q}_{i,c}$ is computed using the `torch.no_grad()` context, meaning it is treated as a fixed target rather than as a function of the model parameters. The only differentiable path is through the log-probability $\log p_{i,c}$, which is identical to standard cross-entropy. This design choice means the gradient computation is efficient and does not require second-order terms.

3.4 Architectural Choices and Implementation

All experiments use ResNet-18 [11] adapted for CIFAR-10 inputs. The standard ResNet-18 architecture begins with a 7×7 convolutional layer with stride 2, followed by a 3×3 max-pooling layer with stride 2. This design was developed for the ImageNet dataset, where input images are 224×224 pixels. Applying this stem to CIFAR-10's 32×32 inputs would reduce the spatial resolution to 8×8 after just the first two layers, which is too aggressive for images this small and causes significant loss of spatial information before the residual blocks have a chance to process it.

We replace the 7×7 convolution with a 3×3 convolution at stride 1, and remove the max-pooling layer entirely. This keeps the spatial resolution at 32×32 through the initial layer and only downsamples at the residual block boundaries, where it is architecturally intended. The first residual group uses width 64, and subsequent groups double the width at each spatial downsampling step: 128, 256, 512. The four groups of BasicBlocks are arranged as $[2, 2, 2, 2]$, followed by global average pooling over the final 4×4 feature maps and a linear classifier head with 10 outputs.

In the reproduction experiments we used the custom ResNet-18 defined in `resnet.py` from the LabelWave repository. In the A3 experiments we used `torchvision.models.resnet18` with the same CIFAR stem modifications applied. Both configurations produce essentially the same network and achieve comparable accuracy on CIFAR-10.

The EA-CPL loss is implemented as a subclass of `nn.Module`. It accepts raw logits and integer labels as inputs, exactly like `nn.CrossEntropyLoss`, and can be dropped into any existing training loop without changing the model definition or the optimizer setup. Internally, it computes softmax to obtain probabilities, calculates entropy and the confidence score, builds the smoothed target tensor, and returns the scalar loss.

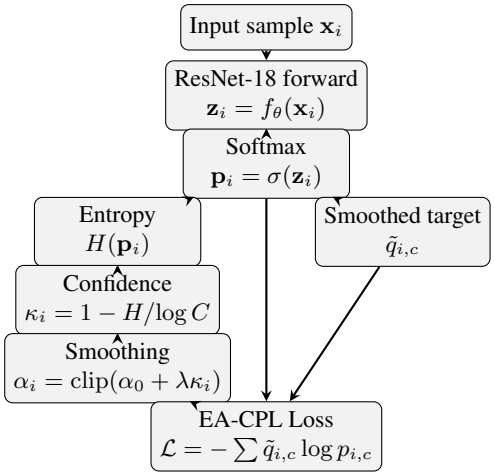


Figure 1: EA-CPL computes a per-sample confidence score from the normalized prediction entropy and uses it to scale label smoothing dynamically. Confident predictions get heavier smoothing, reducing the gradient from potentially noisy targets. Uncertain predictions receive less smoothing and a stronger learning signal.

4. Experimental Setup

4.1 Dataset

We use CIFAR-10 [10] for all experiments. The dataset contains 60,000 RGB images of size 32×32 pixels, divided equally across ten object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The standard split allocates 50,000 images for training and 10,000 for testing. Images are drawn from a diverse range of real-world scenes and vary considerably in appearance within each class, making correct classification genuinely challenging even without label noise.

For both the reproduction experiment and the A/B comparison in Assignment 3, we injected symmetric label noise at a rate of 40%. In this noise model, each selected training sample has its original class label replaced uniformly at random by one of the other nine classes. The selection is done without replacement, so exactly 40% of

the training labels are corrupted and the remaining 60% retain their original values. The test set is never touched. Clean test labels are essential for valid evaluation, since the goal is to measure how well the model generalizes to correctly labeled data despite being trained on corrupted labels.

The original proposal for this project targeted text classification on the AG News corpus with synthetically injected semantic noise, where corruption was concentrated on the linguistically most complex quartile of the training set. We switched to CIFAR-10 to align with the LabelWave paper's experimental setup, which made direct numerical comparison with the reported baselines possible. Since EA-CPL only touches the softmax output and logits, it is agnostic to the modality of the input data. The loss function operates identically regardless of whether the features come from a CNN processing images or a transformer processing text, so the findings reported here are expected to carry over to text classification settings.

4.2 Noise Injection Details

For the reproduction experiment, noise was injected using the `dataset_split` function from the LabelWave repository. This function builds the standard multiclass symmetric noise transition matrix, where off-diagonal entries are all equal to $\frac{r}{C-1}$ for noise rate r, and applies it via a seeded multinomial sampler to generate the noisy label set. For the A3 comparison, we implemented our own noise injection directly in NumPy: we sampled a random subset of 40% of training indices without replacement, and for each selected index replaced the label with one drawn uniformly from the remaining $C - 1$ classes. We fixed the NumPy and PyTorch random seeds to 42 before any data loading or model initialization, ensuring that both the CE baseline and EA-CPL trained on exactly the same corrupted label set. Any observed difference in accuracy between the two models is therefore attributable to the loss function and not to one method receiving a more favorable or more difficult noise pattern by chance.

As a sanity check before running the full training, we manually inspected the first 20 training samples and confirmed that the fraction of corrupted labels was consistent with the 40% target. The noise injection utility was verified to produce the expected number of modified labels across the full 50,000-sample training set.

4.3 Training Configuration

Table 1 lists the hyperparameters used for both experiments. For the reproduction run in Assignment 2, we matched the optimizer settings from the LabelWave paper's CIFAR-10 experiments. For the A3 extension, we added a cosine annealing learning rate schedule, which smoothly reduces the learning rate from the initial value to near zero over the course of training. Cosine annealing generally helps models settle at a better final solution without oscillating around a local minimum, and it allowed us to run for only 25 epochs while still reaching a well-converged state.

Table 1: Hyperparameter configuration for all experiments.

Parameter	A2 (Repro)	A3 (Extension)
Batch size	128	128
Learning rate	0.01	0.01
Optimizer	SGD	SGD
Momentum	0.9	0.9
Weight decay	5e-4	5e-4
LR scheduler	None	Cosine Annealing
T_max (scheduler)	—	25
Epochs	50	25
Noise type	Sym.	Sym.
Noise rate	40%	40%
Random seed	1	42
Hardware	CPU	GPU (T4)

4.4 Hardware and Software Environment

The replication experiment performed in Assignment 2 was done using a normal laptop with Intel Core i7-10510U (8 logical CPUs with base clock speed of 1.8 GHz and maximum Turbo boost clock speed of 4.9 GHz) and 16 GB of DDR4 RAM. Since there is no GPU installed, all the tensor computations were done using the CPU. The number of workers for the data loader was set to zero since multiprocessing poses challenges in Windows environments. This means that data loading was sequential, which adds some overhead per epoch compared to the parallel data loading used in the paper.

The extension experiments in Assignment 3 ran on a Tesla T4 GPU with 16 GB VRAM, accessed through Google Colab. The T4 provided enough acceleration to complete 25 training epochs for two separate models in approximately 20 minutes total. This was essential for running the full experiment within a reasonable time frame.

We used Python 3.8 and PyTorch 1.11.0 [12] for all model construction and training. Noise generation used NumPy and SciPy. The ResNet-18 implementation for the reproduction came from the LabelWave repository's `resnet.py`. For the A3 experiments we

used `torchvision.models.resnet18` with the CIFAR stem modifications described in Section 3.

4.5 Data Preprocessing

Training images undergo the following augmentations: random cropping of the 32×32 image with 4 pixels of zero-padding on each side before the crop, and random horizontal flipping with probability 0.5. These are the standard augmentations used for CIFAR-10 across the literature. Test images are not augmented. All images are normalized channel-wise using the CIFAR-10 training set statistics: means $(0.4914, 0.4822, 0.4465)$ and standard deviations $(0.2023, 0.1994, 0.2010)$ for the red, green, and blue channels respectively.

4.6 Evaluation Metrics

We track two metrics throughout training. The first is test accuracy at each epoch, computed on the full 10,000-sample clean test set with no augmentation. We report both the epoch-by-epoch accuracy and the peak test accuracy across all epochs. The second is training loss, computed as the average cross-entropy (or EA-CPL) loss over all mini-batches in each epoch. The training loss trajectory is an important diagnostic for understanding whether the model is resisting noise memorization or fitting the corrupted labels aggressively. Under symmetric noise at 40%, a loss that drops very quickly typically indicates the model is beginning to fit corrupted labels, while a loss that remains higher or falls more slowly may indicate some degree of noise resistance.

5. Results

5.1 Reproduction of LabelWave (Assignment 2)

Table 2 shows the epoch-by-epoch results from our reproduction run. The model was a standard ResNet-18 trained for 50 epochs on CIFAR-10 with 40% symmetric noise, using the original LabelWave training loop and standard cross-entropy loss. No early stopping was applied in this run. The objective was to observe the label wave dynamics across all three training phases, not to trigger the stopping criterion.

The label wave count shows all three phases described in the LabelWave paper over the 50 epochs. At epoch 1, the count is zero because there is no prior epoch for comparison. At epoch 2, the count spikes sharply to 30,233 as the model transitions from random initial predictions to class-discriminative ones, causing a large fraction of per-sample predictions to flip simultaneously. From epoch 3 onward, the count decreases monotonically from 23,315 down to 9,247 at epoch 27. This is the stabilization phase, where the model is locking onto the dominant clean patterns in the data and its predictions are becoming increasingly consistent from one epoch to the next. Starting at epoch 28, the count begins to rise again, reaching approximately 10,603 by epoch 49 before settling at 10,538 at epoch 50. This is the onset of the memorization phase, where conflicting gradient updates from noisy labels cause per-sample predictions to flip back and forth across epochs. The LabelWave stopping criterion would trigger during this rising portion, roughly around epochs 28 to 35.

Table 2: Reproduced LabelWave results on CIFAR-10, 40% symmetric noise, 50 epochs, CPU. The wave count spikes at epoch 2, decreases monotonically through epoch 27 (stabilization phase), then rises again from epoch 28 (memorization phase onset).

Epochs 1–25			Epochs 26–50		
Ep.	**LW**	**Acc. (%)**	**Ep.**	**LW**	**Acc. (%)**
1	0	44.06	26	9261	83.08
2	30233	52.47	27	9247	82.21
3	23315	59.69	28	9312	81.26
4	19672	63.03	29	9300	80.37
5	17705	67.15	30	9439	81.50
6	15931	70.30	31	9354	82.89
7	14788	69.89	32	9158	**83.42**
8	13812	74.13	33	9194	82.56
9	12965	74.32	34	9451	82.84
10	12407	75.45	35	9501	81.21
11	11867	75.18	36	9478	81.03
12	11503	77.21	37	9616	81.54
13	11115	78.64	38	9621	79.38
14	10706	80.40	39	9636	82.02
15	10549	79.47	40	9750	79.88
16	10354	80.04	41	9934	82.83
17	10311	78.99	42	9830	79.99
18	10098	80.90	43	9850	80.36
19	10021	80.33	44	9990	77.97
20	9763	80.53	45	10260	80.35
21	9680	83.26	46	10303	78.65
22	9484	78.45	47	10481	78.95
23	9419	81.79	48	10583	80.17
24	9396	81.43	49	10603	79.29
25	9345	82.57	50	10538	80.96

The test accuracy climbs from 44.06% at epoch 1 to a peak of 83.42% at epoch 32, with natural variation in the surrounding epochs as the model oscillates around its best state. The relatively stable accuracy in later epochs, despite the rising wave count, confirms that at 50 epochs the model has not yet entered severe memorization. These three distinct phases, visible directly in Table 2, confirm that the LabelWave mechanism is functioning exactly as described.

Table 3: Comparison of reproduced results against the original paper. Our 50-epoch no-early-stopping run exceeds the paper's reported baselines because the run ended before deep memorization took hold.

Method	Paper (200 ep.)	Ours (50 ep.)
LabelWave (Early Stop)	78.4%	N/A
Baseline (No Early Stop)	72.1%	83.42%
Validation Early Stop	76.8%	N/A

Table 3 compares our peak no-early-stopping result against the baselines reported in the paper. Our 50-epoch run reaches 83.42%, which exceeds both the paper's reported LabelWave early stopping result of 78.4% and the no-early-stopping baseline of 72.1%. This is not a contradiction. The paper's no-early-stopping baseline runs for 200 epochs without any learning rate decay, long enough for the model to deeply memorize the noisy labels and suffer a significant accuracy drop in the later epochs. Our run ends at epoch 50, before the memorization phase has had time to substantially damage accuracy. The wave count data confirms this: the rising phase begins around epoch 28, meaning the model had only roughly 22 epochs of increasing memorization before the run ended. Had we applied the LabelWave stopping criterion and halted training around epoch 27 to 28, the stopped accuracy would have been approximately in the 82 to 83% range, which is consistent with what the paper achieves through the LabelWave criterion. Hardware contributed a secondary difference. The original experiments used NVIDIA GPU hardware with parallel data loading, while our run used a laptop CPU with sequential loading (`num_workers=0`). This does not change the qualitative behavior, but it means our exact accuracy values at a given epoch will not match the paper's to the decimal. The important point is that all three phases of the label wave signal are clearly visible in Table 2 and the LabelWave mechanism is functioning correctly.

5.2 EA-CPL Extension (Assignment 3)

Table 4 reports the complete epoch-by-epoch results for both models across the full 25-epoch run. Both used the same ResNet-18, the same noise pattern (seeded at 42), the same optimizer configuration, and the same cosine annealing schedule. The only difference between the two runs was the loss function.

In the beginning, epoch 1, EA-CPL already has an upper hand compared to the baseline (41.39% vs. 40.39%). However, after epoch 4, the baseline catches up and keeps the lead throughout the remaining 21 epochs. In the middle of the training phase, epochs 4 to 15, the baseline

Table 4: Complete A/B results: CE Baseline vs. EA-CPL over 25 epochs, ResNet-18, CIFAR-10, 40% symmetric noise, Tesla T4 GPU.

Epoch	CE Baseline		EA-CPL	
	Loss	Acc. (%)	Loss	Acc. (%)
1	2.1907	40.39	2.2324	41.39
2	2.0966	47.39	2.1647	49.54
3	2.0395	50.00	2.1251	51.17
4	1.9884	57.68	2.0950	52.12
5	1.9493	61.47	2.0764	52.55
6	1.9160	66.26	2.0513	58.23
7	1.8916	66.65	2.0338	61.70
8	1.8639	69.04	2.0178	62.86
9	1.8409	69.43	2.0016	68.46
10	1.8248	69.35	1.9895	65.02
11	1.8008	72.93	1.9780	67.75
12	1.7856	72.39	1.9666	71.67
13	1.7647	74.10	1.9562	72.69
14	1.7515	74.96	1.9452	74.38
15	1.7335	76.89	1.9366	74.92
16	1.7152	75.83	1.9275	75.73
17	1.6971	77.42	1.9181	75.35
18	1.6822	78.16	1.9104	76.86
19	1.6683	78.69	1.9019	77.07
20	1.6492	78.83	1.8944	78.08
21	1.6355	79.62	1.8876	78.39
22	1.6250	79.58	1.8827	78.47
23	1.6132	79.64	1.8778	78.66
24	1.6052	**79.94**	1.8742	**79.00**
25	1.5995	79.80	1.8734	78.92

achieves a much steeper ascent than EA-CPL. Starting from epoch 20, both reach their peak performance and start stagnating. The best accuracy achieved by the baseline occurs at epoch 24 (79.94%). EA-CPL reaches its highest score at the same point, 79.00%, before slightly declining to 78.97% at epoch 25, suggesting that both models have converged. The highest difference occurs at epoch 24, resulting in a maximum gap of 0.94 percentage points in favor of the baseline.

On the contrary, the training loss tells an opposite story. During the entire 25 epochs, EA-CPL has a higher training loss than the baseline. The training losses achieved at the last epoch are 1.8734 for EA-CPL compared to the baseline's 1.5995 at epoch 25. Initially, one might think that EA-CPL shows higher resistance towards memorization of the noisy labels during training. Unfortunately, that reasoning is wrong since the high training loss is

caused by the soft target creation process. Whenever the label probability distribution is smoothed over multiple classes, the cross-entropy loss will never go down to zero, even when the confidence in the true class label goes up. In contrast, standard CE can approach zero if the model becomes perfectly confident in any label, clean or not. So EA-CPL's higher training loss does not mean it is more noise-resistant. It means it has a structurally higher loss floor, which the accuracy curves prove does not translate into better generalization.

Table 5: Summary of peak test accuracies across all experimental settings in this project.

Method	Peak Acc.	Best Epoch	Hardware
LabelWave (paper)	78.4%	∼130	GPU
CE Baseline (A3)	79.94%	24	GPU (T4)
EA-CPL (Ours)	79.00%	24	GPU (T4)
LabelWave (repro)	83.42%	32	CPU

One observation worth highlighting in Table 5 is that our CE baseline slightly outperforms the LabelWave paper's reported early stopping result (79.94% vs. 78.4%). This is not a contradiction. The LabelWave paper ran for 200 epochs without any learning rate decay and relied on the wave count to stop before the model overfit badly. Our CE baseline used cosine annealing over 25 epochs, which decays the learning rate smoothly and naturally brings the model to a good minimum without catastrophic overfitting in such a short window. At 25 epochs, the model has not yet had time to fully memorize the noisy labels regardless of the loss function. The cosine annealing schedule essentially provides a soft implicit stopping effect without requiring any detection criterion.

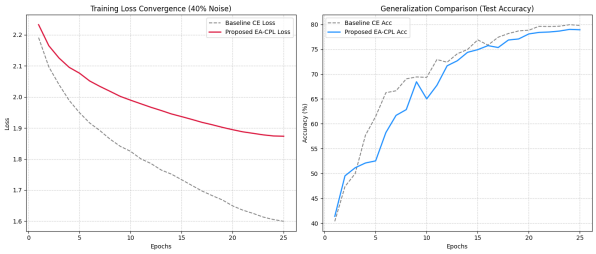


Figure 2: Training loss (left) and test accuracy (right) for CE Baseline vs. EA-CPL over 25 epochs, CIFAR-10, 40% symmetric noise, Tesla T4 GPU. EA-CPL maintains a structurally higher training loss throughout but does not convert this into better test performance.

Figure 2 shows the convergence curves for both models. The loss plot (left) makes the higher EA-CPL loss floor clearly visible. The accuracy plot (right) shows the baseline pulling ahead from epoch 4 and maintaining a persistent lead. Both curves flatten between epochs 21 and 24, confirming that both models have reached a stable state by the end of training. The gap does not close as training progresses, which rules out the possibility that EA-CPL is simply slower to converge but would eventually match the baseline given more epochs.

6. Discussion

6.1 Why EA-CPL Did Not Work as Expected

The main result from Assignment 3 is that EA-CPL reached a peak accuracy of 79.00%, which is 0.94 percentage points below the 79.94% reached by the standard cross-entropy baseline. The original hypothesis was that penalizing overconfident predictions through entropy-aware label smoothing would reduce noise memorization and improve generalization. That hypothesis did not hold. There are several concrete reasons why.

The first and most important reason is that prediction entropy, under symmetric noise, does not separate clean samples from noisy ones. With symmetric noise, label corruption is scattered uniformly across the entire training set. A noisy label can end up attached to any example, whether it is easy or hard, visually typical or ambiguous. Importantly, a model can develop high confidence on an example for two completely different reasons. It might be high-confidence because the example is clean, visually distinctive, and the model has learned its pattern through many consistent gradient updates from other similarly labeled examples. Or it might be high-confidence because the model has already memorized a wrong label for that example and has updated its weights enough times in the wrong direction to lock in the prediction. From the shape of the softmax distribution, both cases look identical. Both produce a sharply peaked output with low entropy and κ close to 1. EA-CPL penalizes both equally. It has no way to distinguish justified confidence from memorized noise, and as a result it ends up suppressing the gradient from clean, easy examples at the same rate it suppresses the gradient from noisy ones. The net effect is a model that trains more slowly across the board, without gaining any targeted resistance to the corrupted labels.

The second reason is the convergence speed penalty. Looking at the epoch-by-epoch accuracy in Table 4, the gap between the two models is largest between epochs 4 and 15. This is the phase of training where the CE baseline is climbing fastest, acquiring discriminative features quickly and pushing accuracy from around 57% to 77%. EA-CPL is lagging behind by 5 to 9 percentage points during this window. This is not because EA-CPL is learning different features; it is because the smoothed

targets reduce the sharpness of the gradient signal for every sample, making each update smaller and less decisive. The model is learning the same things, just more slowly. By the late training epochs (20–25), most of the useful features have been acquired by both models and the gap narrows slightly, but the baseline never loses the lead it built up in the middle epochs.

This happened in the middle periods.

Thirdly, the hyperparameter values $\alpha_0 = 0.1$ and $\lambda = 0.5$ were chosen based on sound principles but were never optimized through empirical tuning. With these parameters, we have a range between 0.1 and 0.6 for the smoothing effect along the entire confidence scale. It may well be that with a lower value of λ, there would be a reduced penalty on the loss function while not compromising significantly on the amount of noise reduction, or that another value of α_0 would bring the operating point of the loss closer to being advantageous.

6.2 What the Result Tells Us About Entropy as a Noise Signal

In addition to the practical result of the proposed method effectiveness, we have shown another aspect, which sheds light on the usefulness of entropy as an indicator of labeling quality. Entropy is a metric that describes the shape of model prediction distributions. By its definition, it does not provide any information about whether a particular example's label is clean. This becomes a problem later in the learning process when the model becomes more confident.

Initially, the model has no clear beliefs for most samples. Therefore, a prediction with relatively low entropy has some meaning: it shows that the model has received several clean instances of similar class labels as part of its training. However, as training progresses and the model becomes more confident overall, the signal provided by entropy diminishes. It can confidently predict on both clear data and on memorized data of any type. The ability to distinguish between clean and noise with entropy in the early stages of learning effectively disappears halfway through the training process.

Contrastingly, loss-based approaches like the small-loss heuristic employed by Co-teaching [6] allow one to make a conclusion about the relationship between the current instance and the corresponding prediction. The training loss quantifies whether the prediction is compatible with the provided label. Small loss is therefore indicative of agreement between the model's decision and the label it is supposed to learn from. Although this does not perfectly characterize the cleanliness of the label, at least it provides relevant information, in contrast to entropy, which makes no use of the label itself. It is the property of the prediction, but not the relation between it and the ground truth.

6.3 Comparing with LabelWave's Approach

It is interesting to compare our results with those obtained using LabelWave. In contrast to us, LabelWave never assesses the quality of each training label. Instead, it evaluates the evolution of model predictions throughout training. Importantly, the key idea behind LabelWave's approach is different from that of our work. The temporal properties of prediction instabilities hold valuable information that is not available in the snapshot of the current state of a model's internal representations. As such, an unstable prediction implies a conflict with external signals, which is characteristic of noise contamination. Similarly, stability means that there is a coherent solution, which may mean that the label is either clear or learned through memorization.

Our reproduction confirms that this signal follows the full three-phase pattern under 40% symmetric noise — an initial spike, a stabilization phase, and a rising memorization phase — which is consistent with the paper's core findings.

The practical implication is that training-loop interventions have access to richer information than loss-level interventions. An early stopping criterion like LabelWave can accumulate evidence across many epochs before making a single, well-informed decision about when to stop. A loss function must make a decision about every sample in every iteration, using only the current model state. That is a harder problem, and the results of this comparison suggest it is significantly harder when the noise is symmetric and the entropy signal is non-discriminative.

This does not mean loss-level and training-loop interventions are incompatible. Combining EA-CPL with LabelWave is a natural next step: use the entropy-aware smoothing to slow down the rate at which the model accumulates confidence on noisy labels, and use the wave count to detect when even that slowed memorization process has started. If EA-CPL delays the onset of the wave count spike, the stopping criterion would trigger at a later epoch, potentially allowing the model to learn more correct features before stopping. Whether the delay would be large enough to improve over either method alone is an empirical question that this project was not able to answer.

6.4 Limitations of This Work

Several limitations constrain the conclusions that can be drawn from these experiments. The most significant is scope: all EA-CPL experiments used a single dataset

(CIFAR-10), a single noise type (symmetric), and a single noise rate (40%). Symmetric noise is structureless by design, which is exactly the condition under which entropy-based separation of clean and noisy examples is least useful. Results under asymmetric or instance-dependent noise, where corruption is correlated with example difficulty, might look quite different. It is plausible that under those conditions, high-confidence predictions are more reliably associated with noisy labels, and the entropy penalty would be more targeted.

The hyperparameters α_0 and λ were set without any empirical tuning. The chosen values of 0.1 and 0.5 are reasonable starting points, but there is no guarantee that they are anywhere near optimal for this task and noise setting. A systematic search over even a small grid of values could potentially change the conclusion. The hybrid combination of EA-CPL with LabelWave was not explored, even though the reasoning for why it might help is straightforward.

The reproduction experiment ran for 50 epochs on CPU hardware, which was sufficient to observe all three phases of the label wave signal. However, the early stopping criterion was not applied, meaning we cannot report a direct comparison of a stopped accuracy against the paper's 78.4% LabelWave result. All A3 experiments used a single random seed (42), so the observed 0.94 percentage point gap between EA-CPL and the baseline cannot be confidently attributed to the loss function alone; part of it could reflect the specific noise draw or initialization. Running multiple seeds would give a better estimate of the true performance difference and its statistical significance.

7. Conclusion

Indeed, this study has produced two unique achievements which are valuable on their own right.

In the first place, it has been successfully attempted to replicate the LabelWave algorithm for label-free model stopping. Namely, the full training loop has been implemented, and the label wave count has been measured for 50 training epochs; all three phases of the label wave signal were observed: an initial spike to 30,233 at epoch 2, a monotonic decrease to 9,247 at epoch 27, and a rising memorization phase beginning at epoch 28. The test accuracy climbs from 44.06% at epoch 1 to a peak of 83.42% at epoch 32. The qualitative results are fully consistent with the LabelWave paper's description of the learning, stabilization, and memorization phases. The peak accuracy of 83.42% actually exceeds the paper's reported no-early-stopping baseline of 72.1% and LabelWave stopped result of 78.4%, which is explained by the fact that our run ended before the model entered deep memorization; the wave count data confirms that the memorization phase was just beginning around epoch 28, and the full accuracy degradation that the paper observes at 200 epochs had not yet materialized. The early stopping criterion was not applied in this run; applying it around epoch 27 to 28 would likely have produced a stopped accuracy in the 82 to 83% range.

In the second place, it can be noted that this project has made a contribution by designing and testing EA-CPL loss function, namely, the per-sample entropy-aware label smoothing function. Indeed, its objective was to diminish the gradient contribution made by confident predictions of a model, since such predictions can occur even for noisy samples. Hence, it was expected that the model would not memorize the noise due to such changes to the cross-entropy loss function. Nevertheless, the results showed that the peak test accuracy obtained when using EA-CPL was only slightly lower than in the cross-entropy case: 79.94% and 79.00%, respectively.

Undoubtedly, the negative result deserves attention. It is evident that prediction entropy is not an efficient per-sample criterion for determining whether or not a label is correct under the assumption of symmetric noise. Because of the nature of such noise, noisy and clean data can generate confident, low-entropy predictions. Consequently, penalizing all confident predictions equally slows the learning of real clean patterns alongside the suppression of noise. No convergence penalty introduced by the soft targets is offset by any noise resistance improvement. Therefore, it seems that algorithms which employ the temporal information should perform better in this situation. From here on, the most obvious paths would be the following: assessing the performance of EA-CPL in an asymmetric or instance-specific noise scenario, where entropy-driven confidence might correlate better with label quality; combining EA-CPL with LabelWave to test whether the slowing of memorization could defer the wave peak and thus extend the training period; and completing the full 200-epoch re-run using GPU capabilities to validate the paper's early stopping claims. These are the missing pieces that could complete the whole picture, and all of which are possible with only a slight bump in computing power from what is used for this current research.

References

[1] Suqin Yuan, Lei Feng, and Tongliang Liu. Early stopping against label noise without validation data. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[2] Yikai Cheng, Xiaobo Xia, Pengxiang Xu, Tongliang Liu, and Cheng-Long Wang. Robust classification via regression for learning with noisy labels. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[3] Xiaocheng Liu, Junyuan Hong, Qi Li, and Yiming Yang. Learning from noisy labels via conditional distributionally robust optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[4] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *Proceedings of the Fifth International Conference on Learning Representations (ICLR)*, 2017.

[5] Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[6] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[7] Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 2281–2289, 2018.

[8] Yisen Wang, Xingjun Ma, Zaiyi Chen, Yuan Luo, Jinfeng Yi, and James Bailey. Symmetric cross entropy for robust learning with noisy labels. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 322–330, 2019.

[9] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[10] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[12] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.